QA Automation Framework & Test Case Sample

Carlo Tano — Software QA Engineer · yantano.github.io/carlo-tano-portfolio · tano.carlom@gmail.com

This document summarizes a Playwright/TypeScript end-to-end test automation framework built for my own portfolio site (yantano.github.io/carlo-tano-portfolio), along with a sample of the manual test cases the automated specs are based on. The full, runnable source code is available in a public repository — see the link at the end of this document.

1. Framework Overview

The framework follows the Page Object Model (POM) pattern with custom Playwright fixtures, written in TypeScript. It targets a real, live website rather than a toy example, so every spec maps to an actual user flow: theme switching, the contact form, an interactive AI chat widget, outbound project links, and responsive layout across breakpoints.

Layer	Tool / Approach
Test runner	Playwright Test
Language	TypeScript
Design pattern	Page Object Model + custom fixtures
Browsers / devices	Chromium, Firefox, WebKit, Pixel 7 emulation, iPhone 14 emulation
Accessibility	axe-core (WCAG 2 A/AA automated scan)
CI/CD	GitHub Actions — on push, on PR, and a nightly scheduled run against the live URL
Reporting	Playwright HTML report, trace + video on failure, uploaded as a CI artifact

2. Project Structure

```
playwright-portfolio-tests/
|-- tests/
|   |-- pages/                  # Page Object Model
|   |   |-- HomePage.ts         # navbar, theme switch, hero, back-to-top
|   |   |-- ContactSection.ts   # contact form
|   |   `-- AiWidget.ts         # Carlo AI chat widget
|   |-- fixtures/
|   |   `-- test-fixtures.ts    # wires page objects into `test`
|   `-- specs/
|       |-- navigation.spec.ts
|       |-- theme-toggle.spec.ts
|       |-- contact-form.spec.ts
|       |-- ai-widget.spec.ts
|       |-- project-links.spec.ts
|       |-- responsive.spec.ts
|       `-- accessibility.spec.ts
|-- .github/workflows/playwright.yml
|-- playwright.config.ts
|-- package.json
`-- tsconfig.json
```

3. Key Design Decisions

Page Object Model over inline selectors

Locators and interactions live in tests/pages/. Specs only describe behavior, so a selector change means editing one file instead of every spec that touches it.

Fixtures over repeated setup

homePage, contactSection, and aiWidget are injected automatically and already navigated, so each spec starts at the interesting part rather than repeating boilerplate.

Deterministic network stubbing where it matters

The contact-form spec intercepts the EmailJS call so the suite is fast and repeatable and does not send real emails on every run — while a separate spec still makes real HTTP requests to every outbound project link to catch genuinely broken URLs.

Testing independent state explicitly

The site has two separate theme systems — a site-wide light/dark switch and a scoped theme toggle inside the AI chat widget. One spec explicitly asserts that toggling one does not leak into the other, since that is an easy regression to introduce silently.

4. Sample Page Object

Excerpt from tests/pages/HomePage.ts, showing locators and helper methods used across multiple specs:

```
export class HomePage {
  readonly themeSwitchInput: Locator;
  readonly profilePhoto: Locator;

  constructor(page: Page) {
    this.themeSwitchInput = page.locator('#input');
    this.profilePhoto = page.locator('#profilePhoto');
  }

  async isDarkMode(): Promise {
    return !(await this.page.locator('body')
      .evaluate(el => el.classList.contains('site-light-mode')));
  }

  async toggleTheme() {
    await this.themeSwitchInput.click();
  }
}
```

Corresponding spec (tests/specs/theme-toggle.spec.ts):

```
test('defaults to dark mode on first visit', async ({ homePage }) => {
  await expect(homePage.themeSwitchInput).toBeChecked();
  expect(await homePage.isDarkMode()).toBe(true);
});
```

5. Automated Coverage Summary

- Navigation — section scrolling, active-link state, mobile menu, back-to-top, resume download link
- Theme switch — dark-by-default, persistence across reload, background/photo swap, fresh-session default
- Contact form — required-field and email-format validation, success and error states
- AI chat widget — open/close, scoped theme toggle, clear chat, click-away-to-close
- Project cards — link integrity (target/rel), live HTTP status checks, broken image detection
- Responsive — 4 breakpoints (375 / 768 / 1440 / 1920px), no horizontal overflow, no console errors
- Accessibility — axe-core WCAG 2 A/AA scan, alt-text presence, keyboard reachability

6. Sample Manual Test Cases

A subset of the manual test cases the automated specs above are derived from, formatted in the style used for test case documentation / Jira test tickets.

ID	Title	Steps	Expected Result	Priority
TC-001	Theme switch defaults to dark mode	1. Open site in a fresh/incognito session 2. Observe the theme switch and background	Switch shows checked/moon state; dark particle background and carlo-dark.jpeg photo are shown	**High**
TC-002	Theme preference persists across reload	1. Toggle switch to light mode 2. Refresh the page	Site reloads in light mode: sky background, carlo-light.jpeg photo, switch unchecked	**Medium**
TC-003	Contact form blocks empty submission	1. Scroll to Contact section 2. Click Send without filling any field	Form does not submit; Name field receives focus via native HTML5 validation	**High**
TC-004	Contact form rejects invalid email	1. Fill Name, Subject, Message 2. Enter 'not-an-email' in Email field 3. Click Send	Email field is flagged invalid; form does not submit	**High**
TC-005	Contact form success path	1. Fill all fields with valid data 2. Click Send 3. Wait for response	Success note 'Message sent successfully!' is shown; form fields reset	**High**
TC-006	AI widget theme is isolated from site theme	1. Open Carlo AI widget 2. Toggle the widget's own theme button 3. Check the site background	Widget switches between its light/dark skin; the site-wide background and photo are unaffected	**Medium**
TC-007	Outbound project links are safe and valid	1. Open Projects section 2. Inspect each Live Demo / GitHub link	Every link has target='_blank' and rel='noopener'; every URL responds with a non-4xx/5xx status	**Medium**
TC-008	No horizontal overflow at any breakpoint	1. Load the page at 375px, 768px, 1440px, and 1920px widths	document.documentElement.scrollWidth never exceeds clientWidth at any tested width	**Medium**

Full source code, CI configuration, and the complete spec suite: **github.com/YanTano/carlo-tano-portfolio-tests**.